



实验二：MySQL数据库安全性实验

井明

数据科学与计算机学院

jingming@sdu.edu.cn

2025年5月11日



山东女子学院
Shandong Women's University

实验目的

- 理解MySQL数据库中常见的安全威胁。
- 掌握MySQL用户权限管理的方法。
- 学会使用防止SQL注入的基本技术。（选作）
- 掌握数据加密存储的基础操作。
- 掌握MySQL日志审计功能的基本使用。

实验模块与内容

模块编号	模块名称	实验内容概述	涉及命令/技术
1	用户权限管理	创建用户、授予/撤销权限、验证权限边界	CREATE USER, GRANT, REVOKE, SHOW GRANTS
2	SQL 注入演示与防护	使用拼接 SQL 漏洞语句模拟注入攻击；使用预处理语句防护	SQL 拼接漏洞、prepare()、bind_param()
3	数据加密存储	使用 AES 加密邮箱字段，SHA2 加密用户密码	AES_ENCRYPT(), AES_DECRYPT(), SHA2()
4	日志与审计分析	启用通用查询日志和慢查询日志，分析可疑行为	MySQL 配置文件编辑、general_log, slow_query_log

关键知识点总结

-  **最小权限原则**：用户只授予完成任务所需的最小权限。
-  **SQL 注入防护**：应使用预处理语句或 ORM 框架防止注入。
-  **数据加密存储**：敏感数据如邮箱、密码应加密存储，防止泄露。
-  **日志审计功能**：开启查询日志和慢查询日志有助于溯源和问题排查。

实验内容

一、MySQL用户权限管理

1. 创建一个普通用户，并授予部分权限：

```
CREATE USER 'testuser'@'localhost' IDENTIFIED BY 'Test@123';  
GRANT SELECT, INSERT ON school_db.* TO 'testuser'@'localhost';
```

2. 验证用户权限：

- 用 testuser 登录，测试是否可以修改、删除数据。
- 测试是否可以访问非授权的数据库。

3. 回收权限并重新分配：

```
REVOKE INSERT ON school_db.* FROM 'testuser'@'localhost';
```

二、SQL注入测试与防护 (选作)

1. 使用简单的Web前端或命令行模拟以下场景：

```
SELECT * FROM users WHERE username = '$input' AND password = '$pass';
```

2. 尝试用 ' OR '1'='1 等方式绕过登录验证，演示注入风险。

3. 编写防注入SQL示例（使用预处理语句）：

```
$stmt = $conn->prepare("SELECT * FROM users WHERE username=? AND password=?");  
$stmt->bind_param("ss", $username, $password);
```

三、数据加密存储实验

1. 使用MySQL内置函数进行简单加密存储:

```
INSERT INTO secure_table (username, email, password)
VALUES ('user1', AES_ENCRYPT('user1@example.com', 'key123'), SHA2('pass123', 256));
```

2. 读取并解密数据:

```
SELECT username, AES_DECRYPT(email, 'key123') AS email FROM secure_table;
```

四、MySQL日志与审计

1. 开启通用查询日志和慢查询日志，查看MySQL日志文件位置：

```
[mysqld]
general_log = 1
general_log_file = /var/log/mysql/mysql.log
slow_query_log = 1
long_query_time = 2
slow_query_log_file = /var/log/mysql/mysql-slow.log
```

2. 使用日志排查未授权访问或潜在攻击。

实验思考题

1. 如何设计权限最小化原则下的数据库账户体系？
2. SQL注入漏洞在实际开发中如何避免？
3. 数据加密与性能之间如何权衡？
4. 审计日志如何帮助发现安全问题？

实验环境建议

- MySQL 8.0
- Linux 或 Windows 环境
- 可选：PHP+Apache（用于SQL注入演示），或使用简单Python Flask模拟注入

实验要求

1. 根据实验内容和实验环境，完成实验目的。
2. 记录实验过程，并形成实验报告。

参考答案

一、用户权限管理部分

-- 创建实验数据库

```
CREATE DATABASE IF NOT EXISTS school_db;  
USE school_db;
```

-- 创建数据表

```
CREATE TABLE students (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(50),  
    grade INT  
);
```

-- 插入测试数据

```
INSERT INTO students (name, grade) VALUES  
('Alice', 90), ('Bob', 78), ('Charlie', 85);
```

-- 创建新用户

```
CREATE USER 'testuser'@'localhost' IDENTIFIED BY 'Test@123';
```

-- 授予SELECT和INSERT权限

```
GRANT SELECT, INSERT ON school_db.* TO 'testuser'@'localhost';
```

-- 可选: 查看权限

```
SHOW GRANTS FOR 'testuser'@'localhost';
```

二、SQL注入演示（PHP 示例）

1. 漏洞版登录脚本（vulnerable_login.php）

```
<?php
$conn = new mysqli("localhost", "root", "yourpassword", "school_db");

$username = $_GET['username'];
$password = $_GET['password'];

$sql = "SELECT * FROM users WHERE username='$username' AND password='$password'";
echo "执行的SQL: $sql <br>";

$result = $conn->query($sql);
if ($result->num_rows > 0) {
    echo "登录成功";
} else {
    echo "登录失败";
}
?>
```

“ 输入用户名为 admin' -- 就能绕过验证

2. 安全版（使用预处理语句）

```
<?php
$conn = new mysqli("localhost", "root", "yourpassword", "school_db");

$username = $_GET['username'];
$password = $_GET['password'];

$stmt = $conn->prepare("SELECT * FROM users WHERE username=? AND password=?");
$stmt->bind_param("ss", $username, $password);
$stmt->execute();

$result = $stmt->get_result();
if ($result->num_rows > 0) {
    echo "登录成功";
} else {
    echo "登录失败";
}
?>
```

三、加密存储示例

```
-- 创建加密存储表
CREATE TABLE secure_users (
  id INT PRIMARY KEY AUTO_INCREMENT,
  username VARCHAR(50),
  email VARBINARY(255),
  password_hash VARCHAR(255)
);
```

```
-- 插入数据, 使用AES和SHA2
INSERT INTO secure_users (username, email, password_hash)
VALUES (
    'user1',
    AES_ENCRYPT('user1@example.com', 'key123'),
    SHA2('pass123', 256)
);
```

```
-- 查询并解密
SELECT
    username,
    CAST(AES_DECRYPT(email, 'key123') AS CHAR) AS email,
    password_hash
FROM secure_users;
```

四、日志与审计配置（Linux 环境）

1. 修改 MySQL 配置文件 my.cnf 或 mysqld.cnf

```
[mysqld]
general_log = 1
general_log_file = /var/log/mysql/mysql.log

slow_query_log = 1
long_query_time = 2
slow_query_log_file = /var/log/mysql/mysql-slow.log
```

2. 重启 MySQL:

```
sudo systemctl restart mysql
```

五、附加表：users（用于登录测试）

```
USE school_db;

CREATE TABLE users (
    id INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50) UNIQUE,
    password VARCHAR(50)
);

INSERT INTO users (username, password) VALUES
('admin', 'admin123'),
('user1', 'pass123');
```